



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G4

System Integration and Architecture

System Design Document (SDD)

Project Title: RevnU: A Sales Management System for
Restaurants

Prepared By: Bigno, Richemmae Valle

Version: 1.0

Date: 02/18/2026

Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	02/18/2026	Richemmae Bigno	Initial draft	Draft

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY.....	4
1.0 INTRODUCTION.....	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION.....	5
3.0 NON-FUNCTIONAL REQUIREMENTS.....	9
4.0 SYSTEM ARCHITECTURE.....	11
5.0 API CONTRACT & COMMUNICATION.....	12
6.0 DATABASE DESIGN.....	17
7.0 UI/UX DESIGN.....	18
8.0 PLAN.....	21

EXECUTIVE SUMMARY

1.1 Project Overview

RevnU is a restaurant management system designed to digitize and streamline daily financial and operational tracking for small and medium-sized food establishments. The system enables restaurant owners to record sales, expenses, and staff salaries while automatically computing daily and monthly profits.

The system is composed of a Spring Boot backend API, a React web application, and an Android mobile application, all integrated through RESTful APIs to provide consistent and secure data access across platforms.

1.2 Objectives

1. Develop a functional restaurant management system focused on financial tracking and daily operations
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android Kotlin (mobile)
3. Design secure and maintainable RESTful APIs for cross-platform communication
4. Provide a simple, intuitive interface suitable for non-technical users
5. Deploy the system to a cloud-based environment for production and real usage
6. Enforce security through JWT authentication, BCrypt password hashing, and Role-Based Access Control (RBAC)
7. Integrate features including external public APIs, Google OAuth login, file uploads, and SMTP email notifications.

1.3 Scope

Included Features:

- User authentication via OAuth 2.0 (Google Sign-In) with JWT support
- User registration and login functionality.
- Minimum of 2 user roles (e.g., Admin and Staff) with API and UI-level restrictions.
- Daily sales recording with categories and variable pricing
- Expense tracking with profit computation
- Staff management and salary payout recording

- Full CRUD operations for at least one major business entity.
- Daily and monthly financial summaries
- End-of-day closing to finalize and lock daily records
- File upload capability for images or PDFs linked to database records.
- SMTP email sending for account-related and system notification purposes.
- React web application for full management
- Android mobile app for sales and expense entry
- Spring Boot REST API built with a layered architecture.
- PostgreSQL database
- Cloud deployment and Git version control

Excluded Features:

- Customer ordering / POS
- Advanced Inventory tracking
- Payment processing
- Exporting data to PDF or CSV formats.
- Advanced Analytics dashboards
- Push Notifications
- Offline mode capability
- Multi-branch support

1.0 INTRODUCTION

1.1 Purpose

This document defines the system design and requirements for RevnU, a restaurant management application intended to support small food establishments that rely on manual or semi-manual methods for tracking sales, expenses, and staff compensation. It serves as a reference for system functionality, architecture, and implementation scope.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RevnU

Domain: Restaurant Management / Small Business Systems

Primary Users: Restaurant Owners

Problem Statement: Many small restaurants and food businesses rely on manual record-keeping methods such as notebooks or spreadsheets to track daily sales, expenses, and staff salaries. These approaches are time-consuming, error-prone, and make it difficult to generate accurate financial summaries or assess business performance over time.

Solution: RevnU provides a centralized digital platform that automates financial calculations, organizes daily records, and presents clear summaries to support informed decision-making.

2.2 Core User Journeys

Journey 1: Daily Sales Recording

1. User logs into the system
2. Navigates to the sales module
3. Records daily sales with category and amount
4. System updates daily totals automatically

Journey 2: End-of-Day Closing

1. User reviews sales and expenses for the day
2. Initiates end-of-day closing
3. System validates entries
4. Daily records are finalized and locked
5. Net profit is generated

Journey 3: Staff Salary Management

1. User accesses staff management
2. Adds or updates staff records
3. Records salary payouts
4. System updates expense and profit totals

Journey 3: Expense Management with File Upload

1. User accesses the expense module to record a new cost.
2. User enters the expense details and uploads a digital copy of the receipt (image/PDF).
3. System stores the file on the server and links it to the specific database record.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration, login, and Google OAuth 2.0 social login.
2. JWT Authentication with BCrypt password hashing and a `/me` endpoint
3. Role-Based Access Control (RBAC) with at least Admin and Staff roles.
4. Full CRUD operations for Sales, Expenses, and Staff Salary entities.
5. External API integration to display meaningful real-world data.
6. SMTP Email sending for account welcomes and system notifications.
7. Sales, expense, and salary tracking
8. File upload and viewing (receipts/documentation).
9. Profit computation
10. End-of-day transaction closing
11. Web dashboard
12. Android mobile app (limited functionality)

SHOULD HAVE

1. Monthly financial summaries
2. Basic validation and error handling
3. Search and filter functionality for sales and expense history.

COULD HAVE

1. Inventory tracking
2. Report exports in PDF or CSV format.

WON'T HAVE

1. POS and payments
2. Advanced analytics
3. Notifications
4. Multi-branch support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me
- **Security:** JWT tokens, password hashing with BCrypt, and secure OAuth flow.

Feature: Role-Based Access Control (RBAC)

- **Screens:** User Management (Admin only), Dashboard (Conditional view).
- **Functions:** Restrict sensitive financial data to Admin; limit entry-only access for Staff.
- **API Endpoints:** GET /admin/users, PUT /admin/roles/{userId}.
- **Security:** API-level role restriction using Spring Security @PreAuthorize.

Feature: Sales Management

- **Screens:** Sales Entry, Sales Summary.
- **Display:** Add, view, and update daily sales records.
- **API Endpoints:** POST /sales, GET /sales?date=, PUT /sales/{id}, DELETE /sales/{id}.
- **Validation:** Mandatory amount and category fields; daily record locking after EOD.

Feature: Expense Management & File Upload

- **Screens:** Expense Entry, Receipt Viewer.
- **Functions:** Add/categorize expenses and upload receipt images or PDFs.
- **API Endpoints:** POST /expenses, GET /expenses, POST /expenses/upload/{id}.
- **Security:** Files linked to database records and stored securely on the server.

Feature: Staff & Salary Management

- **Screens:** Staff List, Salary Log.
- **Data Collected:** Add staff details (name, role), record salary payouts, and upload proof of payment.
- **API Endpoints:** `POST /staff`, `GET /staff`, `POST /salaries`, `POST /salaries/receipt/{id}`.
- **Functions:** Track staff records and link salary expenses to the overall business profit computation.

Feature: End-of-Day (EOD) Closing

- **Screens:** Daily Summary Dashboard.
- **Functions:** Review total sales and expenses, lock daily records to prevent modification, and finalize net profit.
- **API Endpoints:** `POST /day/close`, `GET /day/summary/{date}`.
- **Integration:** Upon closing, the system automatically triggers an **SMTP email** containing the daily financial report to the owner.

Feature: External API Integration

- **Screens:** Dashboard, Sales Summary.
- **Functions:** Consume real-world public API data (e.g., Currency rates for sales conversion).
- **API Endpoints:** `GET /external/rates`.
- **Display:** Show dynamic external data alongside internal restaurant sales data.

Feature: Automated Email Notifications (SMTP)

- **Trigger:** Account creation and End-of-Day closing.
- **Functions:** Send automated Welcome emails and digital daily summary receipts.
- **Security:** Integration with an SMTP server to ensure deliverability.

2.5 Acceptance Criteria

AC-1: Successful User Registration & OAuth Login

Given I am a new user

When I enter a valid email and a strong password, or use the "Sign-In with Google" option

And the system successfully validates the credentials or Google token

Then my account should be created and saved in the database with a hashed password (BCrypt)

AC-2: Daily Sales Entry & Data Linking

Given I am logged in with a valid JWT session

When I record sales for the day with a specific category and amount

Then the system should accurately update daily totals

And the record must be linked to my specific user ID via the `/me` endpoint to track who encoded the data

AC-3: Role-Based Access Control (RBAC)

Given I am logged in with a "Staff" role

When I attempt to access the Admin Dashboard or delete a finalized sale record

Then the system should deny access with a "403 Forbidden" error at both the UI and API levels

AC-4: Expense Tracking with File Upload

Given I have a physical receipt for a business expense

When I create an expense entry and upload an image or PDF of the receipt

Then the file should be stored on the server and linked to that specific expense record in the PostgreSQL database

And I should be able to view or download the file later for verification

AC-5: End-of-Day Closing & Automated Email

Given all sales and expenses for the day are recorded

When the Admin initiates the "End-of-Day Closing"

Then all records for that date should be locked to prevent further editing and the net profit should be calculated

And the system must automatically send a summary notification email (SMTP) to the owner

AC-6: External API Integration

Given I am viewing the Sales Management or Dashboard screen

When the page loads

Then the system should fetch and display real-world data from a public API (e.g., Currency rates or Weather) that provides meaningful context for the day's sales

AC-7: Secure Data Handling (DTO Implementation)

Given the system processes a request for user or financial data

When the API returns a response

Then the system must utilize Data Transfer Objects (DTOs) to ensure sensitive information, such as password hashes, is never exposed

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for CRUD operations.
- End-of-Day (EOD) locking and summary generation ≤ 3 seconds
- Support at least 50 concurrent users
- Database queries ≤ 500 ms
- Caching for External API data to maintain UI fluidity.

3.2 Security Requirements

- HTTPS encryption for all client-server communications.
- JWT-based authentication
- Password hashing using bcrypt
- Role-Based Access Control (RBAC) at API and UI levels
- Input validation and SQL injection prevention
- Mandatory use of DTOs to prevent sensitive data exposure

3.3 Compatibility Requirements

- Web: Chrome, Firefox, Edge (latest versions)
- Android: API Level 24+

- Screen sizes: Mobile, tablet, desktop
- OS: Windows, macOS, Linux

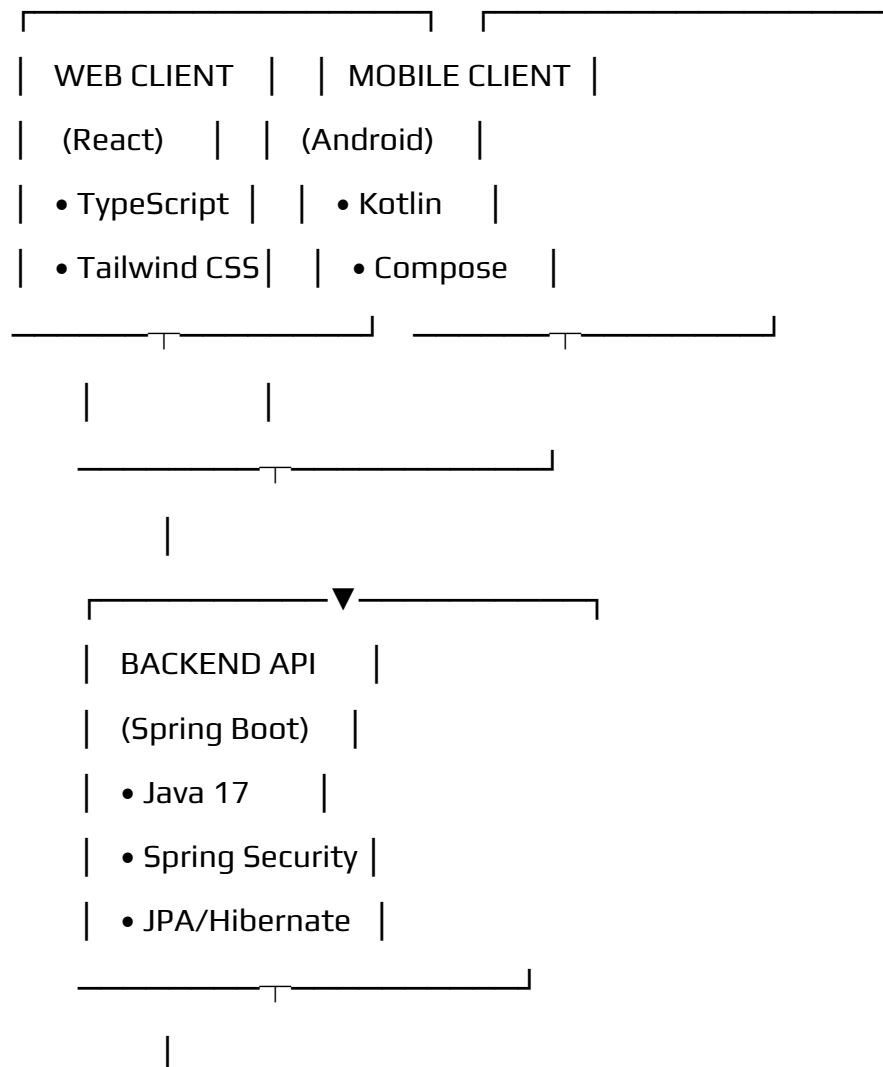
3.4 Usability Requirements

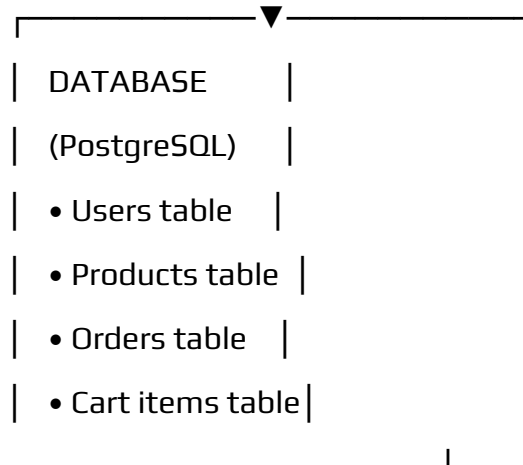
- Minimal and intuitive interface
- Daily tasks completable within minutes
- Clear error and validation messages
- Consistent navigation across all screens

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram





Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Render (Backend), Vercel (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** <https://revnu-api.com/api/v1>
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

json

```
{
  "success": boolean,
  "data": object|null,
  "error": {
```

```
"code": string,  
"message": string,  
"details": object|null  
},  
"timestamp": string  
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

GET /auth/me

HEADER: { Authorization: Bearer {token} }

RESPONSE: { id, email, fullName, role }

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Sales Management Endpoints

POST /sales

BODY: { amount, category, description }

RESPONSE: { success, data: { id, amount, date} }

GET /sales

HEADER: { Authorization: Bearer {token} }

QUERY: ?date=YYYY-MM-DD

RESPONSE: { sales: [{ id, amount, category }] }

Expense & File Management Endpoints

POST /expenses

BODY: { amount, description, category }

RESPONSE: { id, amount, status }

POST /expenses/{id}/upload

BODY: { file }

RESPONSE: { message, fileUrl }

Staff & Operations Endpoints

POST /staff

HEADER: { Authorization: Bearer {token} }

BODY: { fullName, role, salaryRate }

RESPONSE: { success, data: { id, name } }

POST /day/close

HEADER: { "Authorization": "Bearer {token}" }

RESPONSE: { message, totalProfit }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

json

```
{  
  "success": false,  
  "data": null,  
  "error": {  
    "code": "AUTH-001",  
    "message": "Invalid credentials",  
    "details": "Email or password is incorrect"  
  },  
  "timestamp": "2024-01-28T10:30:00Z"  
}
```

```
{  
  "success": false,  
  "data": null,
```



```
"error": {
  "code": "VALID-001",
  "message": "Validation failed",
  "details": {
    "email": "Email is required",
    "password": "Must be at least 8 characters"
  }
},
"timestamp": "2024-01-28T10:30:00Z"
}
```

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

[ERD TO BE INSERTED]

Detailed Relationships:

- **One-to-Many: Roles → Users** (A role can be assigned to multiple users, but each user has one primary role for RBAC).

- **One-to-Many: Users → Sales** (Each sales record tracks the specific user/staff member who encoded it).
- **One-to-Many: Users → Expenses** (Each expense is linked to the user who recorded the transaction).
- **One-to-One: Expenses ↔ Files** (Each expense can optionally be linked to exactly one receipt file upload).
- **One-to-Many: Staff → Salaries** (A staff member can have multiple salary payout records over time)

Key Tables:

1. **users** - Stores account credentials, hashed passwords, and OAuth links.
2. **roles** - Defines the permissions levels (Admin, Staff) for RBAC enforcement.
3. **sales** - Tracks daily revenue, categories, and date-specific metadata.
4. **expenses** - Records business costs and links to digital receipt storage.
5. **files** - Manages metadata for uploaded images/PDFs stored on the server.
6. **staff** - Maintains records of restaurant employees and their basic details.
7. **salaries** - Logs payout history for financial profit computations.

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role_id, oauth_id, created_at.
- **roles:** id, role_name.
- **sales:** id, amount, category, description, user_id, created_at.
- **expenses:** id, amount, description, category, user_id, file_id, created_at.
- **files:** id, file_name, file_path, file_type, uploaded_at.
- **staff:** id, full_name, position, salary_rate, created_at.
- **salaries:** id, staff_id, amount, payment_date, user_id.

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

Phase 1: Planning & Design (Week 1-2)

- **Week 1: Requirements & Architecture**
 - Day 1-2: Project setup and documentation initialization.
 - Day 3-4: Finalization of FRS and NFR including RBAC definitions.
 - Day 5-7: System architecture design (Layered Architecture).
- **Week 2: Detailed Design**
 - Day 1-2: API specification for sales, expenses, and file endpoints.
 - Day 3-4: Database design for 5+ mandatory tables.
 - Day 5-6: UI/UX wireframes for web and mobile.
 - Day 7: Implementation plan finalization.

Phase 2: Backend Development (Week 3-4)

- **Week 3: Foundation**
 - Day 1: Spring Boot setup with Security and JPA dependencies.
 - Day 2: Database configuration, entities, and DTO implementation.
 - Day 3: JWT authentication and BCrypt hashing implementation.
 - Day 4: OAuth 2.0 (Google Login) integration and `/me` endpoint.
 - Day 5: RBAC logic for Admin and Staff role restrictions.
- **Week 4: Core Features**
 - Day 1: Sales and Expense CRUD operations.
 - Day 2: File upload service for receipts.
 - Day 3: External API integration and SMTP email service.
 - Day 4: Staff and salary management logic.
 - Day 5: Global exception handling and API testing.

Phase 3: Web Application (Week 5-6)

- **Week 5: Frontend Foundation**
 - Day 1: React setup with TypeScript and Tailwind CSS.
 - Day 2: Authentication pages and Google OAuth integration.
 - Day 3: Sales entry and summary modules.
 - Day 4: Expense management with file upload UI.
 - Day 5: Staff and Salary management screens.
- **Week 6: Complete Web Features**
 - Day 1: End-of-Day (EOD) closing workflow.
 - Day 2: External API data visualization.
 - Day 3: Admin-only management dashboard (RBAC).
 - Day 4: Responsive design polish.
 - Day 5: Backend integration and unit testing.

Phase 4: Mobile Application (Week 7-8)

- **Week 7: Android Foundation**
 - Day 1: Android Studio setup and project structure.
 - Day 2: Authentication and Google Login screens.
 - Day 3: Sales and Expense input forms.
 - Day 4: File upload integration for mobile camera/gallery.
 - Day 5: API service layer using Retrofit.
- **Week 8: Complete Mobile App**
 - Day 1: Dashboard and EOD summary views.
 - Day 2: Staff salary log access.
 - Day 3: UI polish and Jetpack Compose animations.
 - Day 4: Testing on emulator and physical devices.

- Day 5: APK generation and documentation.

Phase 5: Integration & Deployment (Week 9-10)

- **Week 9: Integration Testing**
 - Day 1: End-to-end testing of SMTP and File uploads.
 - Day 2: Bug fixes for cross-platform data synchronization.
 - Day 3: Security review (JWT/BCrypt/RBAC).
 - Day 4: Performance testing for API response times.
 - Day 5: Final SDD and documentation updates.
- **Week 10: Deployment**
 - Day 1: Backend deployment to cloud.
 - Day 2: Web app deployment.
 - Day 3: Final system verification and APK distribution.
 - Day 4: User manual preparation.
 - Day 5: Final project submission.

8.2 Milestones

- **M1 (End Week 2):** All design documents (SDD, ERD, API specs) complete.
- **M2 (End Week 4):** Backend API fully functional with Security and Integrations.
- **M3 (End Week 6):** Web application complete with full Admin/Staff features.
- **M4 (End Week 8):** Mobile application complete and integrated.
- **M5 (End Week 10):** Full system deployed, integrated, and ready for defense.

8.3 Critical Path

1. **Security Core:** JWT, BCrypt, and Google OAuth setup (Week 3).
2. **Sales & Expense CRUD:** Essential business logic (Week 3-4).
3. **Mandatory Integrations:** File uploads, External API, and SMTP services (Week 4).
4. **Admin Dashboard:** Ensuring RBAC functions across platforms (Week 6).
5. **Cross-platform testing:** Ensuring Web and Mobile sync correctly (Week 9).